

CSE 4403: Algorithms

Lecture03

Sorting Algorithms -2

Anika Farzana

Junior Lecturer

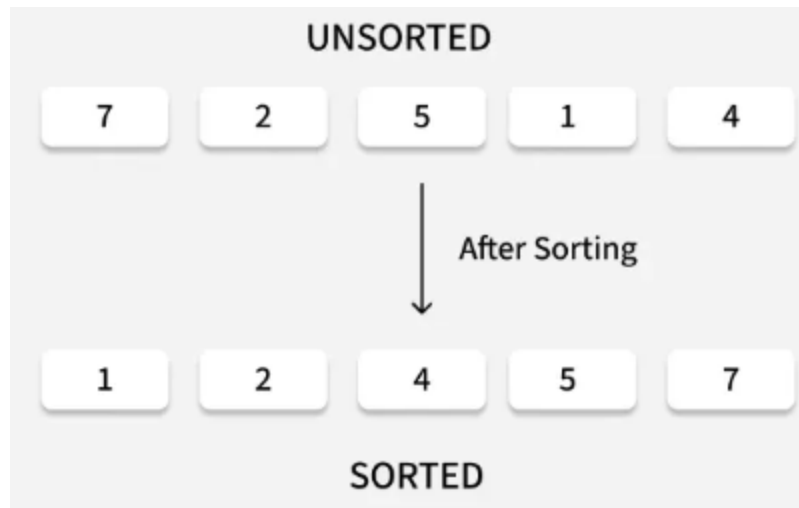
Department of Computer Science & Engineering

Islamic University of Technology, Dhaka, Bangladesh

Email: anikafarzana@iut-dhaka.edu

Sorting

- Rearranging items in an array or list in a specified order: ascending or descending



Sorting Algorithms: Covered

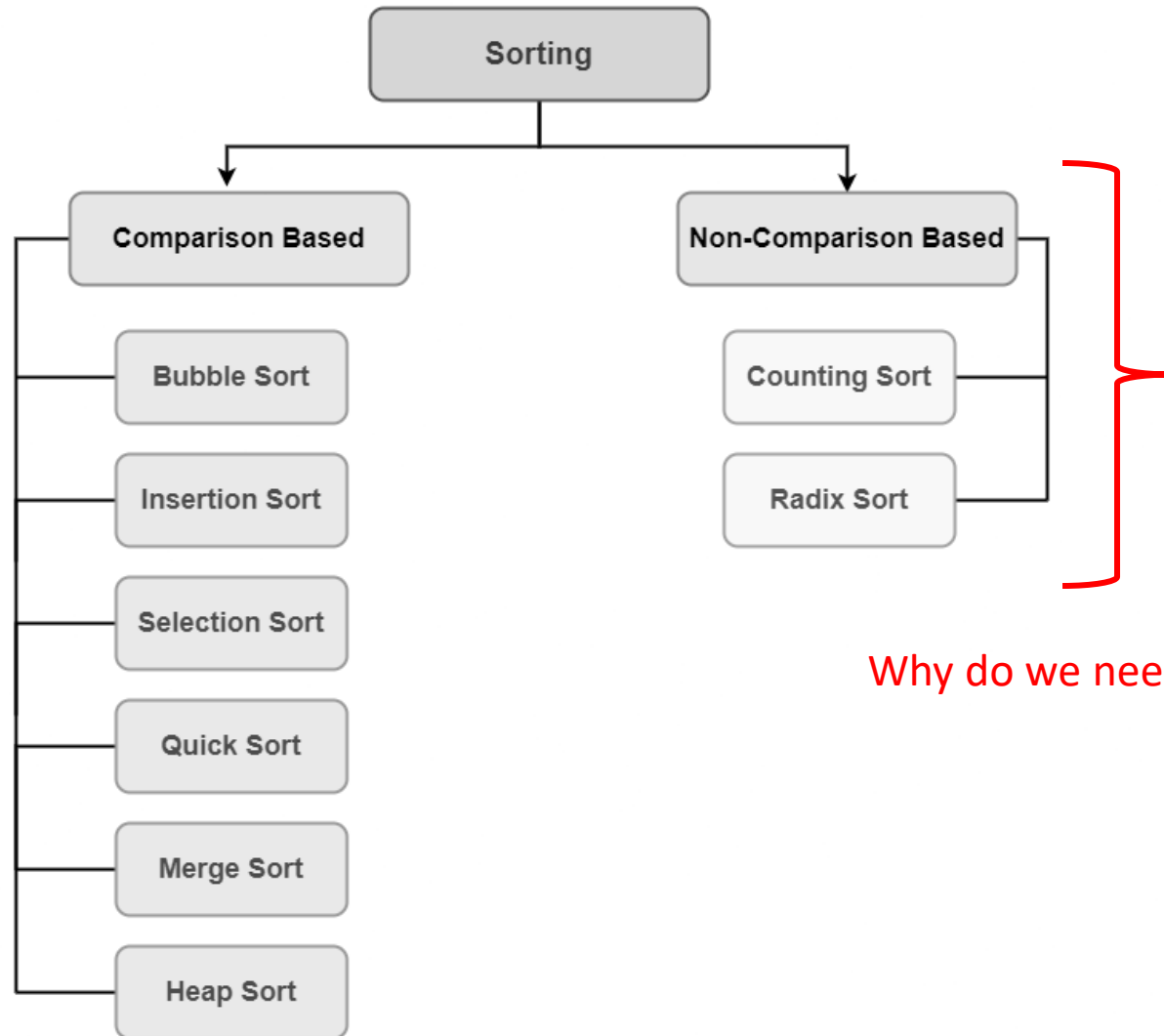
- ✓ Bubble Sort
- ✓ Insertion Sort
- ✓ Selection Sort
- ✓ Counting Sort

Radix Sort

- ✓ Merge Sort

Quick Sort

Types



Why do we need them?

Branching

```
int Smaller(int a, int b)
{
    if(a < b)
        return a;    50%
    else
        return b;    50%
}
```

```
int Smaller_Branchless(int a, int b)
{
    return a*(a<b)
        +b*(b<=a);
}
```

Branching

```
int Smaller(int a, int b)
{
    if(a < b)
        return a;    50%
    else
        return b;    50%
}
```

```
int Smaller_Branchless(int a, int b)
{
    return a*(a<b)
        +b*(b<=a);
}
```

- Modern processors use **branch prediction**.
- They try to guess whether the condition will be true or false.
- If the guess is wrong, the CPU pipeline must be cleared and restarted.
- That wastes time.
- So code with many unpredictable if-else conditions can be slower.

Branching

- Traditional comparison-based sorts like:
 - Bubble Sort
 - Insertion Sort
 - Selection Sort
 - Quick Sort
 - Merge Sort

All depend heavily on comparisons:

`if (arr[j] > key)`

These are branches.

But **Counting Sort** avoids most of that. Instead of asking: “which one is bigger?” it directly does:

`count[arr[i]]++`

Counting Sort

- A **non-comparison-based** sorting algorithm.
- Works efficiently when the **range of input values is small** relative to the number of elements.
- Counts the frequency of each distinct element.
- Uses that count to place elements directly into their correct sorted positions.
- A stable sorting algorithm.

Counting Sort : Code

```
vector<int> countsort(vector<int>& arr) {
    int n = arr.size();

    // find the maximum element
    int maxval = 0;
    for (int i = 0; i < n; i++)
        maxval = max(maxval, arr[i]);

    // create and initialize count array
    vector<int> count(maxval + 1, 0);

    // count frequency of each element
    for (int i = 0; i < n; i++)
        count[arr[i]]++;

    // compute prefix sum
    for (int i = 1; i <= maxval; i++)
        count[i] += count[i - 1];

    // build output array
    vector<int> ans(n);
    for (int i = n - 1; i >= 0; i--) {
        ans[count[arr[i]] - 1] = arr[i];
        count[arr[i]]--;
    }

    return ans;
}
```

Counting Sort : An example

Index: 0 1 2 3 4 5

Array:

1	0	1	2	4	2
---	---	---	---	---	---

Max Val = 4

**Cumulative count array/
Prefix Sum count array:**

0	1	2	3	4
1	3	5	5	6

Index: 0 1 2 3 4 5

Result array:

--	--	--	--	--	--

Counting Sort : An example

Index: 0 1 2 3 4 5

Array:

1	0	1	2	4	2
---	---	---	---	---	---

Max Val = 4

**Cumulative count array/
Prefix Sum count array:**

0	1	2	3	4
1	3	5	5	6

Place 2 in index $5-1=4$

Index: 0 1 2 3 4 5

Result array:

				2	
--	--	--	--	---	--

Counting Sort : An example

Index: 0 1 2 3 4 5

Array:

1	0	1	2	4	2
---	---	---	---	---	---



Max Val = 4

**Cumulative count array/
Prefix Sum count array:**

0	1	2	3	4
1	3	4	5	6



Decrement prefixCnt[2]

Index: 0 1 2 3 4 5

Result array:

				2	
--	--	--	--	---	--

Counting Sort : An example

Index: 0 1 2 3 4 5

Array:

1	0	1	2	4	2
---	---	---	---	---	---



Max Val = 4

**Cumulative count array/
Prefix Sum count array:**

0	1	2	3	4
0	2	3	5	5

Index: 0 1 2 3 4 5

Final sorted array:

0	1	1	2	2	4
---	---	---	---	---	---

Counting Sort : Complexity

Time Complexity:

$O(n + m)$, where n and m are the size of `arr[]` and `count[]` respectively, that is m is the no. of distinct integers allowed.

Auxiliary Space: $O(n + m)$

Counting Sort : A BIG NO!

- For 32-bit integers, you will need a counter array of size $4,294,967,296 = 4\text{GB}$.
- Each element is an integer, so total memory needed = $4 \times 4\text{GB} = 16\text{GB}$.
- For 64-bit integers, you will need $2^{64} * 8 = 138\text{ Exabytes}$.
- World's total memory capacity was approximately, 55(year 2000), 295(year 2007) Exabytes.

Sorting Algorithms

- ✓ Bubble Sort
- ✓ Insertion Sort
- ✓ Selection Sort
- ✓ Counting Sort
- Radix Sort
- ✓ Merge Sort

Quick Sort

Radix Sort

- A non-comparison-based sorting algorithm.
- Used to sort integers and fixed length strings.
- A **Linear sorting algorithm** that sorts elements by processing them digit by digit.
- Uses Counting Sort underneath
- Stable sorting since Counting sort is stable
- Tradeoff between time and space complexity

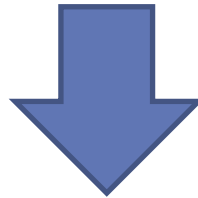
Radix Sort : Main idea

Repeatedly sort the elements by their significant digits, from the least significant to the most significant, achieving the final sorted order.

Radix Sort : An example

Value ≤ 999

5 <u>3</u>	8 <u>9</u>	15 <u>0</u>	3 <u>6</u>	63 <u>3</u>	23 <u>3</u>	17 <u>0</u>	10 <u>2</u>
------------	------------	-------------	------------	-------------	-------------	-------------	-------------

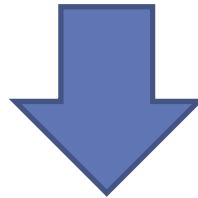


15 <u>0</u>	17 <u>0</u>	10 <u>2</u>	5 <u>3</u>	63 <u>3</u>	23 <u>3</u>	3 <u>6</u>	8 <u>9</u>
-------------	-------------	-------------	------------	-------------	-------------	------------	------------

Radix Sort : An example

Value ≤ 999

1 <u>5</u> 0	1 <u>7</u> 0	1 <u>0</u> 2	<u>5</u> 3	6 <u>3</u> 3	2 <u>3</u> 3	<u>3</u> 6	<u>8</u> 9
--------------	--------------	--------------	------------	--------------	--------------	------------	------------

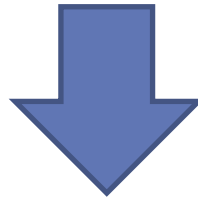


1 <u>0</u> 2	6 <u>3</u> 3	2 <u>3</u> 3	<u>3</u> 6	1 <u>5</u> 0	<u>5</u> 3	1 <u>7</u> 0	<u>8</u> 9
--------------	--------------	--------------	------------	--------------	------------	--------------	------------

Radix Sort : An example

Value ≤ 999

<u>1</u> 02	<u>6</u> 33	<u>2</u> 33	<u>0</u> 36	<u>1</u> 50	<u>0</u> 53	<u>1</u> 70	<u>0</u> 89
-------------	-------------	-------------	-------------	-------------	-------------	-------------	-------------

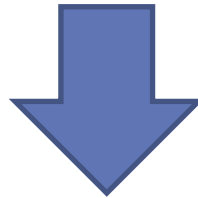


<u>0</u> 36	<u>0</u> 53	<u>0</u> 89	<u>1</u> 02	<u>1</u> 50	<u>1</u> 70	<u>2</u> 33	<u>6</u> 33
-------------	-------------	-------------	-------------	-------------	-------------	-------------	-------------

Radix Sort : An example

Value ≤ 999

<u>1</u> 02	<u>6</u> 33	<u>2</u> 33	<u>0</u> 36	<u>1</u> 50	<u>0</u> 53	<u>1</u> 70	<u>0</u> 89
-------------	-------------	-------------	-------------	-------------	-------------	-------------	-------------



<u>0</u> 36	<u>0</u> 53	<u>0</u> 89	<u>1</u> 02	<u>1</u> 50	<u>1</u> 70	<u>2</u> 33	<u>6</u> 33
-------------	-------------	-------------	-------------	-------------	-------------	-------------	-------------

Sorted !!

Radix Sort : Code

```
void radixsort(int arr[], int n)
{
    // Find the maximum number to
    // know number of digits
    int m = getMax(arr, n);

    // Do counting sort for every digit.
    // Note that instead of passing digit
    // number, exp is passed. exp is 10^i
    // where i is current digit number
    for (int exp = 1; m / exp > 0; exp *= 10)
        countSort(arr, n, exp);
}
```

Radix Sort : Code

```
void countSort(int arr[], int n, int exp)
{
    // Output array
    int output[n];
    int i, count[10] = { 0 };

    // Store count of occurrences
    // in count[]
    for (i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;

    // Change count[i] so that count[i]
    // now contains actual position
    // of this digit in output[]
    for (i = 1; i < 10; i++)
        count[i] += count[i - 1];

    // Build the output array
    for (i = n - 1; i >= 0; i--) {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }

    // Copy the output array to arr[],
    // so that arr[] now contains sorted
    // numbers according to current digit
    for (i = 0; i < n; i++)
        arr[i] = output[i];
}
```

Radix Sort : Space-Time Tradeoff

- Thus, in each step, the size of the count array is equal to the base that we are using. In this case, the **base, b** is 10 ([0-9], at any place/position of the number). The ones place is sorted first, then the tens place, hundreds place and so on.
- The number of iterations of the count Sort is equal to the **maximum number of digits in a number, d**.
- We could have represented the number in other bases, such as the minimum base is 2. In that case, the d would increase. The size of the count array will be equal to the number of digits possible which is only 2, (only 0 and 1). That means, the space complexity is less but the number of iterations of the count Sort increases. In other words, the time complexity is the highest and the space complexity is the least. $(129)_{10} = (10000001)_2$
- A better value of base is often 2^8 or 256 which offers a balance between space and time complexity.

Radix Sort : Complexity

Time Complexity: $O((n + b) * d)$, where b is the base used and d is the maximum number of digits in an element of the array when expressed in the base system b .

In other words, $O(dn)$

Space Complexity: $O(n + b)$. We know for each iteration of the Counting Sort we need $O(n + b)$ auxiliary space at a time.

In other words, $O(n)$